
Computer Organisation and Architecture

Simultaneous Equations Using MARIE Assembly Language

Semester 2

Assigned by: **Dr Razvi Doomun**

Name	Student ID
Yadavee Ramburuth	2512175
Mavish Nuckchady	2518214
Deevesh Moorut	2311422
Doorandharsingh Dookhy	2515500

Contents

1	Introduction and Purpose of the Program	2
2	How Cramer's Rule is Implemented	2
2.1	Main Determinant	2
2.2	Determinants for Each Unknown	2
2.3	Computing the Solutions	3
3	How MARIE Instructions are Used	3
3.1	Input Section	3
3.2	Process Section	3
3.3	Output Section	3
3.4	Error Handling	4
4	Limitations	4
5	Test Cases	4
6	Flowchart	11
7	Reflection and Conclusion	12

1. Introduction and Purpose of the Program

This is a program written in MARIE Assembly Language that solves a system of three simultaneous linear equations in three unknowns using **Cramer's Rule**. The program also include error handling to identify cases where no unique solution exists.

2. How Cramer's Rule is Implemented

Cramer's Rule is a method in linear algebra used to solve systems of linear equations where the number of unknown variables equals the number of equations — in this case, three unknowns and three equations.

Without substitution and elimination, it uses determinants of matrices to solve for the values of the unknown variables directly. In this case, if the system no solution or an infinite number of solutions, the program will return the determinant as zero.

2.1. Main Determinant

The main determinant D is calculated as:

$$D = a_1(b_2c_3 - b_3c_2) - b_1(a_2c_3 - a_3c_2) + c_1(a_2b_3 - a_3b_2)$$

2.2. Determinants for Each Unknown

The determinants for x , y , and z are obtained by replacing the corresponding column in the coefficient matrix with the constant column:

$$D_x = d_1(b_2c_3 - b_3c_2) - b_1(d_2c_3 - d_3c_2) + c_1(d_2b_3 - d_3b_2) \quad (1)$$

$$D_y = a_1(d_2c_3 - d_3c_2) - d_1(a_2c_3 - a_3c_2) + c_1(a_2d_3 - a_3d_2) \quad (2)$$

$$D_z = a_1(b_2d_3 - b_3d_2) - b_1(a_2d_3 - a_3d_2) + d_1(a_2b_3 - a_3b_2) \quad (3)$$

2.3. Computing the Solutions

If $D \neq 0$, the unique solutions are:

$$x = \frac{D_x}{D}, \quad y = \frac{D_y}{D}, \quad z = \frac{D_z}{D}$$

If $D = 0$, the system has no unique solution.

3. How MARIE Instructions are Used

3.1. Input Section

The program uses twelve consecutive INPUT instructions to read all coefficients and constants, entered in the following order:

Input Order			
a	b	c	d
e	f	g	h
i	j	k	l

Each integer is stored in a dedicated memory location using the STORE instruction.

3.2. Process Section

The process section carries out the following steps in order:

1. Calculate the value of the main determinant D .
2. Calculate the determinants D_x , D_y , and D_z .
3. Store all intermediate values in memory.
4. Compute the final values of the three unknowns.

All arithmetic operations are performed using the MARIE instructions LOAD, STORE, ADD, SUBT, and MULT. Careful memory allocation is used to prevent overwriting important values.

3.3. Output Section

When $D \neq 0$, the program calculates the final values of x , y , and z using the DIV instruction and outputs each result using the OUTPUT instruction. Values are displayed in the order: x , then y , then z .

3.4. Error Handling

After calculating D , the program will evaluate whether $D = 0$ using the **SKIPCOND** instruction. If the determinant is 0, the program outputs -1 and stops immediately using the **HALT** instruction. This concerns equations with no unique solution

4. Limitations

- i. The program supports **integer-only arithmetic**, Which do not show the accurate result values of the unknowns, and the values can lose accuracy.
- ii. **Higher memory usage** is required due to the need to store multiple intermediate values.

5. Test Cases

The following table presents all five test cases used to verify the correctness and robustness of the program.

Description	Execution Steps	Input Data	Expected	Actual	Pass
1. Unique Solution ($D \neq 0$)	1. Enter 12 integers 2. Compute D , D_x , D_y , D_z 3. Since $D \neq 0$: compute x , y , z	$a=1, b=1, c=1, d=6$ $e=2, f=5, g=5, h=-4$ $i=2, j=3, k=8, l=5$ $D = 15$	$x=11,$ $y=-5,$ $z=0$	$x=11,$ $y=-5,$ $z=0$	✓
2. Infinite Solutions ($D = 0$)	1. Enter 12 integers 2. Compute determinants 3. Check $D = 0$	$a=1, b=2, c=3, d=4$ $e=2, f=4, g=6, h=8$ $i=4, j=4, k=4, l=4$ $D=0, D_x=0, D_y=0, D_z=0$	Output -1	-1	✓

(continued from previous page)

Description	Execution Steps	Input Data	Expected	Actual	Pass
3. No Solution ($D = 0$, $D_x \neq 0$)	1. Enter 12 integers 2. Compute determinants 3. Check $D = 0$	$a=1, b=1, c=1, d=6$ $e=2, f=2, g=2, h=12$ $i=3, j=3, k=3, l=20$ $D=0, D_x \neq 0$	Output -1	-1	✓
4. Invalid Data	1. Enter non-integer input 2. Program attempts to process	Character input instead of integer	Incorrect result or terminates (no validation)	Undefined behaviour	×
5. No Data Entered	1. Press Enter without values	No values entered	Program halts or behaves unexpectedly	Undefined behaviour	×

Test Case Screenshots

Test Case 1: Unique Solution ($D \neq 0$)

The inputs $a=1, b=1, c=1, d=6, e=2, f=5, g=5, h=-4, i=2, j=3, k=8, l=5$
Determinant $D = 15 \neq 0$. Outputs $x = 11, y = -5, z = 0$.

Inputs

1

Decimal

1

Decimal

1

Decimal

6

Decimal

2

Decimal

5

Decimal

Inputs

5

Decimal

-4

Decimal

2

Decimal

3

Decimal

8

Decimal

5

Decimal

+ Add additional input

File Examples

1 ORG 100

2

3 / =====

4 / INPUT: 12 integers

5 / a1 b1 c1 d1 a2 b2 c2 d2 a3 b3 c3 d3

6 / =====

7 Input

8 Store A1

9 Input

10 Store B1

11 Input

12 Store C1

13 Input

14 Store D1

15

16 Input

17 Store A2

18 Input

19 Store R2

Assemble

Halted

Step

Micro step

Restart

Speed

AC 0000 IR 7000 MAR 200 MBR 7000 PC 201 IN 0005 OUT 0000

+0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F

000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

010 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

020 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

Output log

Decimal

11

-5

0

RTL log

Watch list

Inputs

Display

Test Case 2: Infinite Solutions ($D = 0$)

The inputs $a=1$, $b=2$, $c=3$, $d=4$, $e=2$, $f=4$, $g=6$, $h=8$, $i=4$, $j=4$, $k=4$, $l=4$
 $D = 0$. Since the determinant is 0, the program outputs -1 and halts immediately.

Inputs	Format
1	Decimal
2	Decimal
3	Decimal
4	Decimal
2	Decimal
4	Decimal
6	Decimal
8	Decimal
4	Decimal
4	Decimal
4	Decimal
4	Decimal

Assembly Code:

```

1  ORG 100
2
3  / .....
4  / INPUT: 12 integers
5  / a1 b1 c1 d1 a2 b2 c2 d2 a3 b3 c3 d3
6  / .....
7  Input
8  Store A1
9  Input
10 Store B1
11 Input
12 Store C1
13 Input
14 Store D1
15
16 Input
17 Store A2
18 Input
19 Store B2
20 Input
21 Store C2
22 Input
23 Store D2
24
25 Input
26 Store A3
27 Input

```

Registers: AC: ffff, IR: 7000, MAR: 152, MBR: 7000, PC: 163, IN: 0004, OUT: ffff

Watch List:

Label	Address	HEX	DEC	OCT	BIN
D_NOT_ZERO	163	A000	-24576	120	1000 0000
A1	→ 201	0001	1	1	1
B1	→ 202	0002	2	2	10
C1	→ 203	0003	3	3	11
D1	→ 204	0004	4	4	100
A2	→ 205	0002	2	2	10

Output log: -1

RTI log: PC ← PC + 1
Decoded opcode 0x7 in IR[75-12] as Halt
Halted

Inputs: 1 (Decimal)

Display: 76x16 display, 0xF00-0xFFF

Machine halted normally

Test Case 3: No Solution ($D = 0$, $D_x \neq 0$)

The inputs $a=1$, $b=1$, $c=1$, $d=6$, $e=2$, $f=2$, $g=2$, $h=12$, $i=3$, $j=3$, $k=3$, $l=20$ $D = 0$ with $D_x \neq 0$, confirming no unique solution exists. The program outputs -1 and halts.

The screenshot displays a computer architecture simulator interface. The main window shows assembly code with the following instructions:

```

535 Load QUOT2
536 Jump DIV_OUT
537
538 DIV_NEG, Clear
539 Subt QUOT2
540
541 DIV_OUT, Store DIV_RES
542 Load DIV_RES
543 JumpI DIVS
544
545 DVD, DEC 0
546 DVS, DEC 0
547 DIV_RES, DEC 0
548 D_A, DEC 0
549 D_B, DEC 0
550 REM2, DEC 0
551 QUOT2, DEC 0
552 DS1, DEC 0
553 DS2, DEC 0
554 SRD, DEC 0
555
556 END

```

Below the code, the register file is shown with the following values:

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
010	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
020	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

The right-hand panel shows the 'Output log' with the following content:

```

Decimal
-1

```

The 'Inputs' section shows the following values:

```

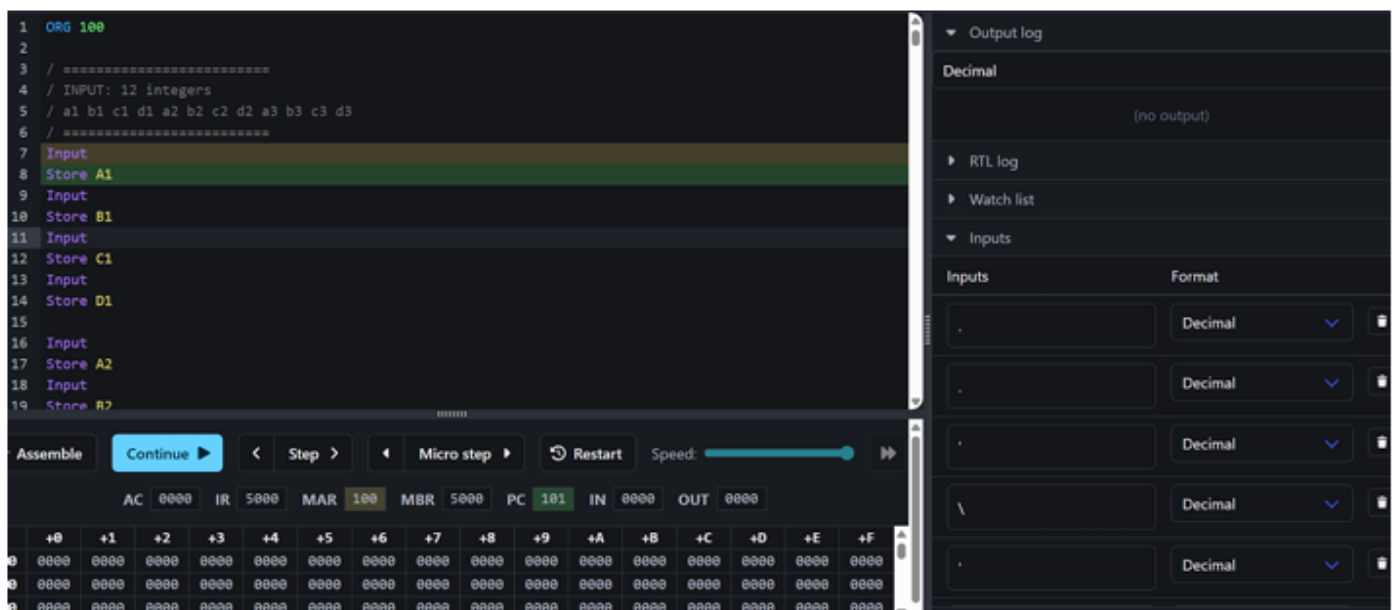
1
1
1
6
2
2
2
12
3
3
3
20

```

The 'Format' section shows the 'Decimal' format selected.

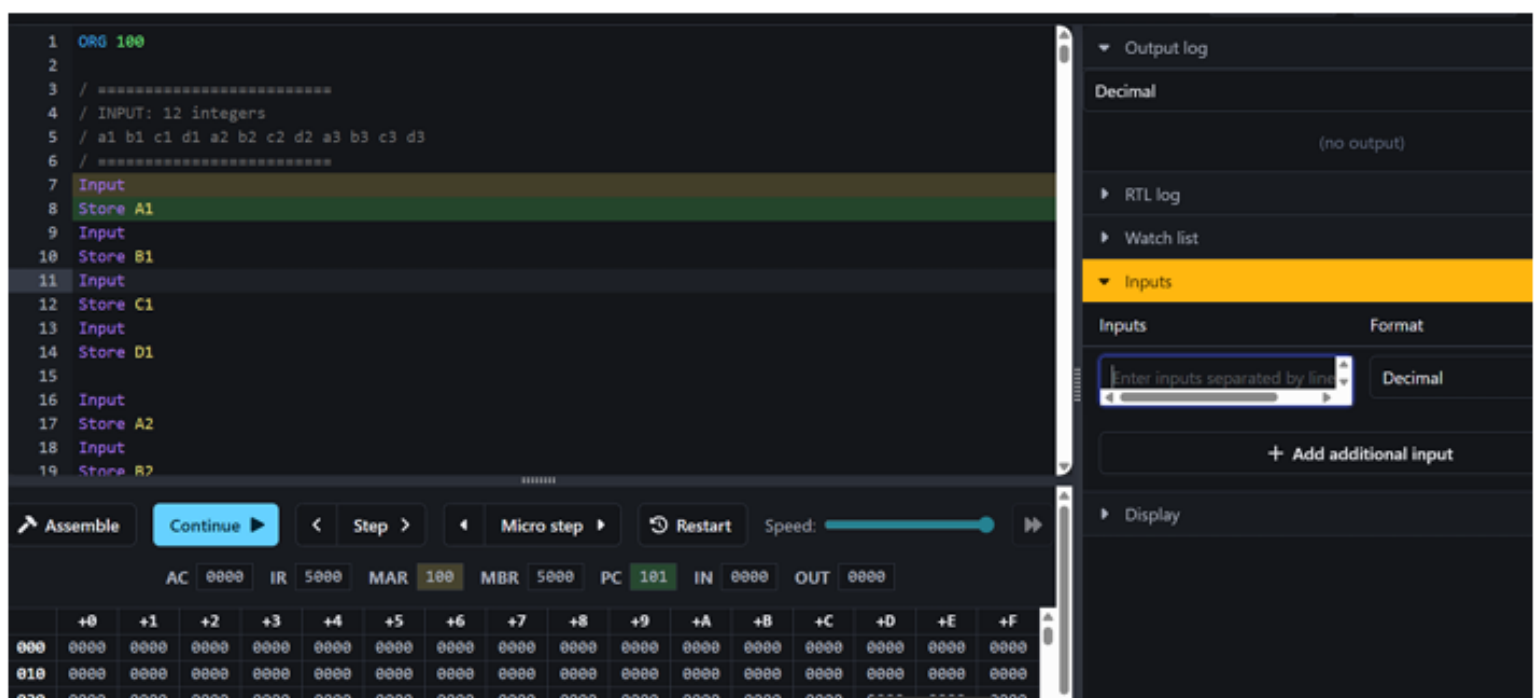
Test Case 4: Incorrect / Invalid Data

When non-integer characters are entered as input, the MARIE simulator has no input validation mechanism. The program attempts to process the data but produces undefined behaviour with no output, as shown below.



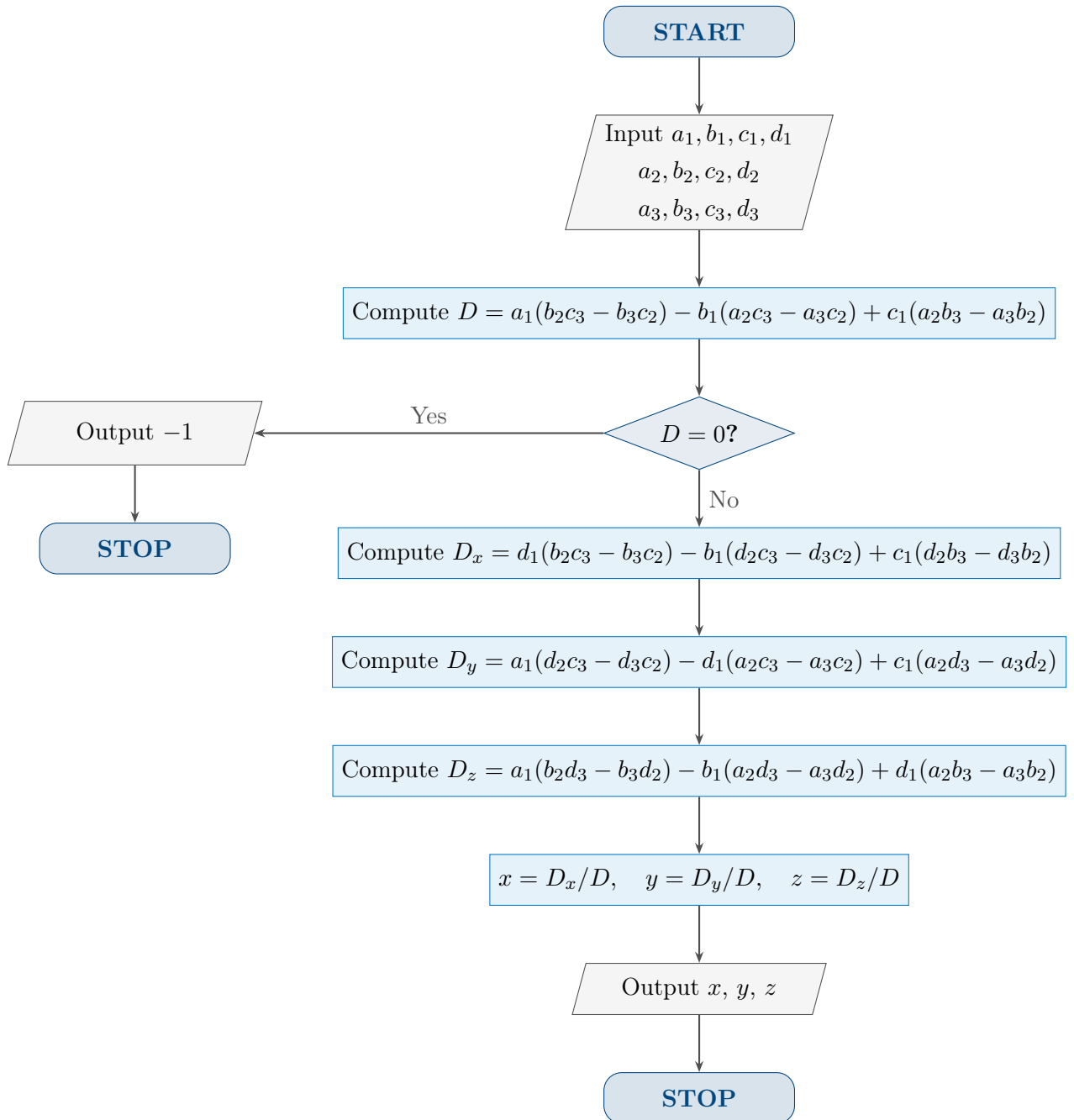
Test Case 5: No Data Entered

When no values are entered and the program is run, the simulator awaits input indefinitely or halts . There is no graceful error message as the program does not include input-presence validation.



6. Flowchart

The flowchart below illustrates the complete logic of the program.



7. Reflection and Conclusion

This program solves 3x3 linear equations system using the Cramer's Rule. Although, it has constraints of the Marie Simulator(supports integer-only arithmetics and high memory usage), it correctly handles arithmetic operations such as multiplication, division and determinant calculation.

One of the main problems during the writing of this program was its length. As the code was prone to errors. Despite these limitations, the program is logically correct and well-structured. Overall, the program code was able to translate the mathematical method into assembly language, showing high-level operations broken down into fundamental machine-level instruction.